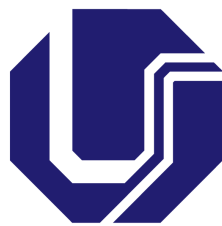

Otimização de Rotas de Patrulhamento com Drones: Um Estudo Comparativo de Meta-heurísticas Bioinspiradas

Heitor Freitas



UFU

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Uberlândia
2026

Heitor Freitas

**Otimização de Rotas de Patrulhamento com
Drones: Um Estudo Comparativo de
Meta-heurísticas Bioinspiradas**

Trabalho de Conclusão de Curso apresentado à
Faculdade de Computação da Universidade Fe-
deral de Uberlândia, como parte dos requisitos
exigidos para a obtenção título de Bacharel em
Ciência da Computação.

Área de concentração: Ciência da Computação

Orientador: Dr. Claudiney Ramos Tinoco

Uberlândia
2026

Aos meus pais, pelo apoio incondicional.

Agradecimentos

Ao Prof. Dr. Claudiney Ramos Tinoco, pela orientação e pelo incentivo ao longo deste trabalho. Aos colegas de curso, pelas discussões e pelo apoio durante a graduação.

“Otimização é a arte de encontrar a melhor resposta possível dentro do que é computacionalmente viável.”

Resumo

Este trabalho investiga o problema de otimização de rotas para patrulhamento com veículos aéreos não tripulados, modelado como uma variante do Problema do Caixeiro Viajante com custo dependente da sequência de visita. O custo de deslocamento entre dois pontos incorpora uma penalização angular calculada a partir do ponto anterior, do ponto atual e do próximo ponto, representado por um tensor tridimensional. Quatro métodos de otimização foram implementados e comparados: busca exaustiva (usada como referência exata em instâncias pequenas), Algoritmo Genético, Otimização por Enxame de Partículas e Otimização por Colônia de Formigas. Os experimentos totalizaram 5029 execuções em 30 instâncias com tamanhos entre 10 e 100 pontos. O Algoritmo Genético encontrou o ótimo global em todas as 18 instâncias pequenas e manteve superioridade nas instâncias maiores, com diferenças de até 22% em relação ao segundo melhor método. Os resultados indicam que a representação direta por permutação e os operadores de cruzamento ordenado são mais adequados a esse problema do que a representação por chaves aleatórias usada pelo PSO ou a construção probabilística tridimensional do ACO.

Palavras-chave: Otimização Combinatória. Problema do Caixeiro Viajante. Meta-heurísticas Bioinspiradas. Patrulhamento com Drones..

Abstract

This work investigates the route optimization problem for unmanned aerial vehicle patrolling, modeled as a variant of the Traveling Salesman Problem with sequence-dependent travel costs. The displacement cost between two points incorporates an angular penalty calculated from the previous, current, and next points, represented as a three-dimensional tensor. Four optimization methods were implemented and compared: exhaustive search (used as exact reference for small instances), Genetic Algorithm, Particle Swarm Optimization, and Ant Colony Optimization. The experiments totaled 5029 runs across 30 instances ranging from 10 to 100 points. The Genetic Algorithm found the global optimum in all 18 small instances and maintained superiority in larger instances, with differences of up to 22% over the second-best method. The results indicate that direct permutation representation with ordered crossover operators is more suitable for this problem than the random-key representation used by PSO or the three-dimensional probabilistic construction of ACO.

Keywords: Combinatorial Optimization. Traveling Salesman Problem. Bio-inspired Metaheuristics. Drone Patrol..

Lista de ilustrações

Lista de tabelas

Tabela 1	– Métodos implementados e parâmetros usados nos experimentos.	36
Tabela 2	– Melhor makespan e gap percentual em relação ao ótimo (busca exaustiva).	37
Tabela 3	– Melhor makespan por instância nas execuções sem busca exaustiva.	38

Lista de siglas

Sumário

1	INTRODUÇÃO	21
1.1	Motivação	22
1.2	Objetivos e Desafios da Pesquisa	23
1.3	Hipótese	23
1.4	Contribuições	23
1.5	Organização do Documento	24
2	FUNDAMENTAÇÃO TEÓRICA	25
2.1	Problema do Caixeiro Viajante	25
2.2	Custo Dependente da Sequência	25
2.2.1	Formulação Matemática	26
2.2.2	Função Objetivo	26
2.3	Roteamento de Drones e Variantes do TSP	26
2.4	Busca Exaustiva	27
2.5	Algoritmos Genéticos	27
2.6	Otimização por Enxame de Partículas	28
2.7	Otimização por Colônia de Formigas	28
2.8	Estudos Comparativos	29
3	PROPOSTA	31
3.1	Modelo Matemático	31
3.1.1	Tensor de Custos	31
3.2	Adaptação dos Métodos	32
3.2.1	Busca Exaustiva	32
3.2.2	Algoritmo Genético	32
3.2.3	Otimização por Enxame de Partículas	32
3.2.4	Otimização por Colônia de Formigas	33
3.3	Desenho Experimental	33

4	EXPERIMENTOS E ANÁLISE DOS RESULTADOS	35
4.1	Método para a Avaliação	35
4.2	Experimentos	36
4.3	Avaliação dos Resultados	36
4.3.1	Instâncias Pequenas: Comparação com o Ótimo	36
4.3.2	Instâncias Maiores: Desempenho Relativo	37
4.3.3	Convergência e Estabilidade	38
4.3.4	Limitações Metodológicas	38
5	CONCLUSÃO	41
5.1	Principais Contribuições	41
5.2	Limitações	42
5.3	Trabalhos Futuros	42
5.4	Contribuições em Produção Bibliográfica	42
	REFERÊNCIAS	43
	 APÊNDICES	 47
	 ANEXOS	 49

Introdução

Veículos aéreos não tripulados (VANTs) oferecem uma alternativa operacional para missões em que pontos de interesse precisam ser visitados com rapidez, regularidade e baixo custo de deslocamento. Em cenários de patrulhamento, inspeção e monitoramento, a tarefa computacional central é definir a ordem de visita desses pontos. Quando se considera um único veículo que parte de uma base, visita todos os pontos uma vez e retorna à origem, a formulação se aproxima do Problema do Caixeiro Viajante (*Traveling Salesman Problem*, TSP), um dos problemas clássicos de otimização combinatória (LAWLER et al., 1985; APPELEGATE et al., 2006).

O TSP é computacionalmente difícil: sua versão de decisão pertence ao conjunto dos problemas NP-completos (GAREY; JOHNSON, 1979). Essa característica torna inviável a enumeração exaustiva de soluções em instâncias de tamanho moderado, pois o número de rotas cresce fatorialmente com a quantidade de pontos. Por esse motivo, heurísticas e meta-heurísticas são empregadas quando o objetivo é obter boas soluções em tempo computacional controlado, ainda que sem garantia de otimalidade global.

A literatura registra dezenas de estudos comparativos entre meta-heurísticas aplicadas ao TSP. Wu (WU, 2020) comparou GA, ACO e PSO em instâncias TSPLIB e observou que o GA apresenta melhor desempenho médio em instâncias de até 100 cidades. Haroun et al. (HAROUN; JAMAL; HICHAM, 2015) relataram vantagem do ACO sobre o GA em termos de qualidade de solução, porém com maior custo computacional. Almufti e Shaban (ALMUFTI; SHABAN, 2025) avaliaram nove meta-heurísticas e concluíram que nenhum método domina todas as classes de instância, em linha com o teorema *No Free Lunch*. Chandra e Naro (CHANDRA; NARO, 2022) aplicaram ANOVA aos resultados de oito métodos e identificaram diferenças estatisticamente significativas entre GA, PSO e ACO. Halim e Ismail (HALIM; ISMAIL, 2019) revisaram seis heurísticas para otimização combinatória e apontaram que a escolha da representação da solução é tão determinante quanto o método de busca. Estudos mais recentes, como o de Hossain e Yilmaz Acar (HOSSAIN; ACAR, 2024), confirmam que a qualidade relativa entre métodos clássicos e modernos depende fortemente do tamanho e da estrutura da instância.

Este trabalho investiga o problema de roteamento para patrulhamento com drones, modelado como uma variante do TSP com custo dependente da sequência de visita. A implementação desenvolvida compara uma busca exaustiva, usada como referência exata em instâncias pequenas, com três meta-heurísticas bioinspiradas: Algoritmo Genético (*Genetic Algorithm*, GA), Otimização por Enxame de Partículas (*Particle Swarm Optimization*, PSO) e Otimização por Colônia de Formigas (*Ant Colony Optimization*, ACO). Todos os métodos otimizam a mesma função objetivo, denominada *makespan*, que representa o tempo total de uma rota iniciada e encerrada na base.

O modelo adotado acrescenta uma particularidade ao TSP euclidiano usual: o custo para seguir de um ponto atual para um próximo depende também do ponto visitado imediatamente antes. Essa dependência permite incorporar uma penalização de curva, compatível com a ideia de que mudanças bruscas de direção afetam o tempo de voo de um drone. Representa-se a instância por um grafo tridimensional de custos, indexado por nó anterior, nó atual e próximo nó.

1.1 Motivação

Aplicações de roteamento com drones mostram que pequenas mudanças no modelo físico ou operacional alteram substancialmente a estrutura do problema. O problema do caixeiro viajante com drone, introduzido por Murray e Chu (MURRAY; CHU, 2015) como *Flying Sidekick Traveling Salesman Problem* (FSTSP), e posteriormente estendido por Agatz, Bouman e Schmidt (AGATZ; BOUMAN; SCHMIDT, 2018) e Dell’Amico et al. (DELL’AMICO; MONTEMANNI; NOVELLANI, 2021; DELL’AMICO; MONTEMANNI; NOVELLANI, 2022), combina decisões de roteamento terrestre e aéreo. Rajan, Sundar e Gautam (RAJAN; SUNDAR; GAUTAM, 2022) abordaram especificamente o roteamento de VANTs em missões de patrulha, propondo um algoritmo de *progressive hedging* para instâncias com horizonte de planejamento finito.

Embora este trabalho não trate de entrega com caminhão e drone, essa literatura reforça a relevância de estudar variantes do TSP quando restrições reais de operação são incorporadas. No caso de patrulhamento aéreo, a ordem de visita não afeta apenas a distância percorrida: a sequência também influencia a suavidade do trajeto, a quantidade de manobras e, por consequência, o tempo estimado de deslocamento. A motivação deste trabalho é avaliar como três meta-heurísticas bioinspiradas se comportam quando aplicadas a essa variante sequencial, usando uma implementação reprodutível e registros estruturados de desempenho.

1.2 Objetivos e Desafios da Pesquisa

O objetivo geral é analisar o desempenho de meta-heurísticas bioinspiradas na otimização de rotas de patrulhamento por drones modeladas como uma variante do TSP com custo dependente da sequência.

Os objetivos específicos são:

- ❑ formalizar a função objetivo usada para avaliar uma rota de patrulhamento com penalização de curva;
- ❑ implementar e comparar busca exaustiva, GA, PSO e ACO sobre o mesmo conjunto de instâncias;
- ❑ medir qualidade da solução, quantidade de avaliações e tempo de execução;
- ❑ analisar a escalabilidade dos métodos quando o número de pontos de interesse aumenta;
- ❑ identificar limitações experimentais e oportunidades de melhoria nos métodos implementados.

1.3 Hipótese

A hipótese principal é que as meta-heurísticas bioinspiradas obtêm rotas de baixo *makespan* com custo computacional inferior ao da busca exaustiva em instâncias nas quais a enumeração completa se torna impraticável. A hipótese secundária é que o comportamento dos métodos difere conforme o tamanho da instância: GA e ACO tendem a explorar diretamente a estrutura permutacional do problema, enquanto PSO depende da qualidade da representação contínua por chaves aleatórias para produzir boas permutações.

As perguntas de pesquisa são:

- ❑ Em instâncias pequenas, quão próximas as soluções das meta-heurísticas ficam da solução ótima obtida por busca exaustiva?
- ❑ Em instâncias maiores, qual método apresenta melhor compromisso entre *makespan* e tempo de execução?
- ❑ As diferenças de convergência e estabilidade entre os métodos explicam seu desempenho relativo?

1.4 Contribuições

As contribuições deste trabalho são:

- ❑ uma formulação computacional de rotas de patrulhamento com custo dependente da sequência anterior–atual–próximo;
- ❑ uma implementação comparativa de quatro métodos de otimização sobre a mesma função objetivo;
- ❑ uma base experimental com saídas estruturadas de resumo, evolução e tempo de execução;
- ❑ uma análise empírica do comportamento de GA, PSO e ACO em instâncias de diferentes tamanhos.

1.5 Organização do Documento

O Capítulo 2 apresenta os conceitos necessários para compreender o problema e os métodos de otimização avaliados. O Capítulo 3 descreve o modelo matemático e as adaptações algorítmicas realizadas. O Capítulo 4 detalha o desenho experimental e a análise dos resultados. O Capítulo 5 reúne as conclusões, limitações e possibilidades de trabalhos futuros.

Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para compreender a formulação do problema e os métodos de otimização comparados neste trabalho. A Seção 2.1 introduz o Problema do Caixeiro Viajante e sua relevância em otimização combinatória. A Seção 2.2 formaliza o modelo de custo dependente da sequência adotado. A Seção 2.3 situa o trabalho no contexto da literatura de roteamento com drones. As Seções 2.4 a 2.7 descrevem os quatro métodos implementados, e a Seção 2.8 sumariza estudos comparativos relevantes.

2.1 Problema do Caixeiro Viajante

O Problema do Caixeiro Viajante (TSP) define a busca por um ciclo de menor custo que visita cada cidade exatamente uma vez e retorna à cidade inicial. O problema ocupa posição central em otimização combinatória por combinar formulação simples, dificuldade computacional elevada e ampla relevância prática (LAWLER et al., 1985; APPLEGATE et al., 2006). A versão de decisão do TSP é NP-completa (GAREY; JOHNSON, 1979), o que explica a inviabilidade da enumeração completa em instâncias grandes.

Em uma instância com n pontos, fixando-se a origem, há $(n - 1)!$ ordens possíveis de visita. Heurísticas clássicas, como a de Lin-Kernighan (LIN; KERNIGHAN, 1973), obtêm soluções de alta qualidade sem examinar todo o espaço de busca. Surveys recentes cobrem o estado-da-arte em variantes do TSP, como o problema generalizado (POP et al., 2024) e aplicações em armazenagem (BOCK et al., 2025). Rajwar, Deep e Das (RAJWAR; DEEP; DAS, 2023) oferecem uma revisão exaustiva de aproximadamente 540 meta-heurísticas para busca e otimização.

2.2 Custo Dependente da Sequência

No TSP clássico, o custo de cada aresta independe do contexto. Em cenários de patrulhamento aéreo, a manobra necessária para mudar de direção ao passar por um

ponto depende de onde o drone veio e para onde vai em seguida. Modelar esse custo exige um tensor tridimensional.

2.2.1 Formulação Matemática

Seja $V = \{0, 1, \dots, n-1\}$ o conjunto de nós, em que 0 representa a base de origem e retorno, e $V' = V \setminus \{0\}$ os pontos de interesse. Define-se o tensor de custos $C \in \mathbb{R}^{n \times n \times n}$, em que cada elemento $c_{i,j,k}$ representa o tempo necessário para deslocar-se do nó j para o nó k sabendo que o nó visitado imediatamente antes de j foi o nó i :

$$c_{i,j,k} = \frac{d(j,k)}{v} + \tau(\vec{v}_{i,j}, \vec{v}_{j,k}), \quad i, j, k \in V \quad (1)$$

em que $d(j,k)$ é a distância euclidiana, v é a velocidade do drone (normalizada para 1), $\vec{v}_{i,j}$ é o vetor de i para j , e τ é a penalização de curva:

$$\tau(\vec{a}, \vec{b}) = \frac{1}{\pi} \cdot \arccos \left(\frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} \right) \quad (2)$$

A penalização τ varia entre 0 (trajetória em linha reta) e 1 (mudança de sentido). Essa formulação segue a modelagem de custos de curva em roteamento (WINTER, 2002; VANHOVE; FACK, 2012), adaptada para o contexto de drones. A literatura de planejamento de trajetória para VANTs reconhece que curvas fechadas aumentam o consumo de energia (AHMED; SHELAMI, 2024).

2.2.2 Função Objetivo

Uma solução é uma permutação $\pi = (\pi_1, \pi_2, \dots, \pi_{n-1})$ dos pontos em V' . O *makespan* $f(\pi)$ é o custo total acumulado:

$$f(\pi) = c_{0,0,\pi_1} + \sum_{t=2}^{n-1} c_{\pi_{t-2},\pi_{t-1},\pi_t} + c_{\pi_{n-2},\pi_{n-1},0} \quad (3)$$

O objetivo é encontrar $\pi^* = \arg \min f(\pi)$. Na implementação, o tensor é armazenado como $G[i][j][k]$, indexado por anterior, atual e próximo.

2.3 Roteamento de Drones e Variantes do TSP

O uso de drones em roteamento motivou novas variantes do TSP. Murray e Chu (MURRAY; CHU, 2015) introduziram o *Flying Sidekick Traveling Salesman Problem* (FSTSP), no qual um caminhão e um drone cooperam para realizar entregas. Agatz, Bouman e Schmidt (AGATZ; BOUMAN; SCHMIDT, 2018) estudaram o TSP com drone por meio de formulações e heurísticas. Trabalhos recentes avançaram modelos exatos para o FSTSP (DELL'AMICO; MONTEMANNI; NOVELLANI, 2022) e variantes com múltiplos drones

(DELL’AMICO; MONTEMANNI; NOVELLANI, 2021). De Freitas e Penna (FREITAS; PENNA, 2020) propuseram uma busca em vizinhança variável para o FSTSP.

Especificamente no contexto de patrulhamento, Rajan, Sundar e Gautam (RAJAN; SUNDAR; GAUTAM, 2022) modelaram o problema de roteamento de VANTs em missões de patrulha como um problema de otimização com horizonte finito, resolvido por *progressive hedging*. Este trabalho se diferencia por modelar o custo de deslocamento com penalização angular dependente da sequência, aproximando o problema de uma rota de patrulhamento em que a suavidade da trajetória afeta o tempo total.

2.4 Busca Exaustiva

A busca exaustiva enumera todas as permutações dos pontos não pertencentes à origem e avalia cada rota pela função objetivo (3). Na implementação, a enumeração usa o algoritmo de Heap para gerar permutações. O método é determinístico e retorna a solução ótima global, mas seu custo cresce como $(n - 1)!$ avaliações. Por isso, a busca exaustiva é usada como linha de base apenas em instâncias pequenas (até 15 pontos).

2.5 Algoritmos Genéticos

Algoritmos Genéticos são meta-heurísticas populacionais inspiradas em seleção e recombinação de indivíduos (HOLLAND, 1975; GOLDBERG, 1989). Potvin (POTVIN, 1996) propôs uma taxonomia dos operadores de cruzamento para TSP em três categorias: preservação de ordem relativa (OX), preservação de posição absoluta (PMX) e preservação de arestas (ERX). Larrañaga et al. (nAGA et al., 1999) apresentaram uma revisão abrangente de representações e operadores, concluindo que a combinação PMX com mutação *swap* ou inversão produz resultados robustos.

Nagata (NAGATA, 2006) propôs o operador *Edge Assembly Crossover* (EAX), que constrói ciclos a partir das arestas de duas soluções pai e os rearranja para gerar filhos. O GA com EAX atinge qualidade comparável à heurística Lin–Kernighan, sendo considerado estado-da-arte em crossover para TSP. Singh, Singh e Chaurasia (SINGH; SINGH; CHAURASIA, 2024) propuseram um híbrido GA-ACO em dois estágios: o ACO gera a população inicial e o GA a refina com operador *Inver-over* e busca local 2-opt.

Na implementação avaliada neste trabalho, cada indivíduo contém uma permutação direta dos pontos. A população inicial é aleatória. A seleção de pais usa torneio, o cruzamento utiliza operador ordenado (OX), a mutação troca dois genes com probabilidade definida, e o elitismo preserva o melhor indivíduo entre gerações.

2.6 Otimização por Enxame de Partículas

A Otimização por Enxame de Partículas foi proposta como meta-heurística populacional baseada na interação entre partículas que ajustam suas posições a partir de componentes de inércia, memória individual e influência social (KENNEDY; EBERHART, 1995). Como o PSO original opera em espaços contínuos, sua aplicação a problemas de permutação exige adaptação.

Clerc (CLERC, 2004) formalizou o PSO discreto para problemas de permutação definindo posições como permutações, velocidade como lista de transposições e os operadores de subtração e adição correspondentes. O mecanismo NoHope/ReHope evita estagnação ao reexpandir o enxame quando não há melhoria. Kappagantula et al. (KAPPAGANTULA et al., 2025) propuseram o DPSO-Q, que integra *reinforcement learning* ao PSO discreto para melhorar a qualidade das soluções em instâncias TSP.

Neste trabalho, o PSO usa representação por chaves aleatórias (*random keys*): cada partícula mantém um vetor real, e a rota é obtida pela ordenação dos índices desse vetor (BEAN, 1994). A cada iteração, a velocidade é atualizada com parâmetros de inércia (w), componente cognitivo (c_1) e social (c_2). A avaliação da sequência resultante atualiza o melhor histórico da partícula e o melhor global do enxame.

2.7 Otimização por Colônia de Formigas

A Otimização por Colônia de Formigas (ACO) modela a construção incremental de soluções por agentes que combinam informação heurística e trilhas de feromônio (DORIGO; MANIEZZO; COLORNI, 1996). O TSP foi uma das primeiras aplicações de referência (DORIGO; GAMBARDILLA, 1997). Dorigo e Stützle (DORIGO; STÜTZLE, 2004) apresentam o framework completo da meta-heurística, cobrindo variantes como *Ant System* (AS), *Ant Colony System* (ACS) e *MAX-MIN Ant System* (MMAS). Blum (BLUM, 2005) oferece uma introdução comparativa das variantes, e Dorigo e Blum (DORIGO; BLUM, 2005) revisam os fundamentos teóricos de convergência.

O MAX-MIN Ant System (MMAS), proposto por Stützle e Hoos (STÜTZLE; HOOS, 2000), introduz limites $\tau_{\min}$ e $\tau_{\max}$ para os valores de feromônio, evitando estagnação. Apenas a melhor formiga deposita feromônio, e os valores são inicializados em $\tau_{\max}$ para promover exploração inicial.

Avanços recentes integram ACO com aprendizado profundo. Ye et al. (YE et al., 2023) propuseram o DeepACO, que utiliza redes neurais de grafos (GNN) treinadas com PPO para gerar heurísticas específicas por instância. O NeuFACO (YE et al., 2025), dos mesmos autores, combina PPO com *Focused ACO*, reconstruindo seletivamente subconjuntos de nós e alcançando gaps de 1–3% em TSPLIB com tempo 60 vezes menor que o DeepACO. Sheppard et al. (SHEPPARD et al., 2024) propuseram o PGACO/PPOACO,

que substitui a atualização de feromônio por gradientes de política com *experience replay*. Liu, Chen e Zhang (LIU; CHEN; ZHANG, 2025) usaram Programação Genética (GP) para projetar automaticamente regras de transição de estado, produzindo regras interpretáveis que competem com heurísticas projetadas manualmente.

Na implementação deste trabalho, cada formiga constrói uma rota a partir da origem usando uma roleta proporcional ao produto entre feromônio e informação heurística. Como o custo depende de três nós, a matriz de feromônio também é tridimensional. Após a construção das rotas, ocorre evaporação global e depósito proporcional a Q/L_k , em que L_k é o *makespan* da rota.

2.8 Estudos Comparativos

O teorema *No Free Lunch* estabelece que nenhum algoritmo de otimização supera todos os demais em todas as classes de problema. Estudos comparativos entre GA, PSO e ACO para TSP confirmam esse princípio. Wu (WU, 2020) avaliou os três métodos em instâncias TSPLIB e observou vantagem do GA em instâncias de até 100 cidades. Haroun et al. (HAROUN; JAMAL; HICHAM, 2015) relataram que o ACO produz soluções de melhor qualidade que o GA, porém com maior tempo computacional. Almufti e Shaban (ALMUFTI; SHABAN, 2025) compararam nove meta-heurísticas e encontraram variação significativa na ordenação conforme a instância.

Chandra e Naro (CHANDRA; NARO, 2022) aplicaram análise estatística (ANOVA e teste de Tukey) a oito métodos, identificando diferenças significativas entre GA, PSO e ACO em nível de 5%. Wadi e Umar (WADI; UMAR, 2025) concentraram-se em abordagens baseadas em enxame e relataram que o PSO apresenta boa escalabilidade, mas perde em qualidade para o GA em instâncias densas. Alexander e Sriwindono (ALEXANDER; SRIWINDONO, 2020) encontraram vantagem do GA sobre o ACO em instâncias pequenas. Hossain e Yilmaz Acar (HOSSAIN; ACAR, 2024) testaram algoritmos clássicos e modernos e concluíram que a diferença de desempenho se reduz em instâncias grandes, enquanto Toaza e Esztergár-Kiss (TOAZA; ESZTERGÁR-KISS, 2023) revisaram 120 meta-heurísticas aplicadas a problemas de scheduling baseados em TSP.

Formulação do Problema e Métodos

Este capítulo formaliza o modelo matemático adotado e descreve como cada método de otimização foi adaptado ao problema de roteamento de patrulhamento com custo dependente da sequência. O modelo baseia-se no Problema do Caixeiro Viajante (TSP), estendido para incorporar penalização angular nas transições entre pontos consecutivos.

3.1 Modelo Matemático

O problema é definido sobre um conjunto de nós $V = \{0, 1, \dots, n-1\}$, em que 0 é a base de origem e retorno. Os demais nós $V' = V \setminus \{0\}$ representam os pontos de interesse (*points of interest*, POIs) a serem visitados exatamente uma vez. Uma rota é uma permutação $\pi = (\pi_1, \pi_2, \dots, \pi_{n-1})$ de V' , e o *makespan* total é calculado conforme a Equação (3).

O tensor de custo $G[\text{anterior}][\text{atual}][\text{próximo}]$ é pré-calculado a partir das coordenadas dos pontos no plano bidimensional. Diferentemente do TSP clássico, em que o custo depende apenas do par origem-destino, aqui cada transição ($j \rightarrow k$) tem seu custo modulado pelo ângulo de chegada ao nó j vindo do nó i . Esse modelo captura o efeito de manobras de drone: mudanças bruscas de direção aumentam o tempo de deslocamento em relação a trajetórias aproximadamente retilíneas.

3.1.1 Tensor de Custos

Para cada terna $(i, j, k) \in V^3$ com $i \neq j$ e $j \neq k$, calcula-se:

$$G[i][j][k] = \frac{d(j, k)}{v} + \frac{1}{\pi} \cdot \arccos \left(\frac{\vec{v}_{i,j} \cdot \vec{v}_{j,k}}{\|\vec{v}_{i,j}\| \|\vec{v}_{j,k}\|} \right), \quad (4)$$

em que $d(j, k)$ é a distância euclidiana, v é a velocidade do drone (normalizada para 1), $\vec{v}_{i,j} = (x_j - x_i, y_j - y_i)$ e $\vec{v}_{j,k} = (x_k - x_j, y_k - y_j)$. Para o primeiro movimento (partida da base sem nó anterior), adota-se $i = 0$ e $\vec{v}_{0,0} = \vec{0}$, resultando em $\tau = 0$ e $G[0][0][\pi_1] = d(0, \pi_1)$. O mesmo se aplica ao retorno: $G[\pi_{n-2}][\pi_{n-1}][0] = d(\pi_{n-1}, 0)$.

Algoritmo 1 Cálculo do makespan

Entrada: permutação π , tensor G ,
 número de pontos n
Saída: makespan f
 $f \leftarrow 0$
 $anterior \leftarrow 0$
 $atual \leftarrow 0$
for $t = 1$ **to** $n - 1$ **do**
 $proximo \leftarrow \pi[t]$
 $f \leftarrow f + G[anterior][atual][proximo]$
 $anterior \leftarrow atual$
 $atual \leftarrow proximo$
end for
 $f \leftarrow f + G[anterior][atual][0]$ {Retorno à base}
return f

O Algoritmo 1 detalha o cálculo do *makespan* para uma permutação π .

3.2 Adaptação dos Métodos

Os quatro métodos compartilham a mesma função objetivo (3) e a mesma representação de instância (tensor G). A seguir descreve-se como cada método foi adaptado ao modelo proposto.

3.2.1 Busca Exaustiva

A busca exaustiva gera todas as $(n - 1)!$ permutações dos POIs usando o algoritmo de Heap. Para cada permutação, calcula-se o *makespan* e registra-se a melhor solução. O método é determinístico e serve como referência exata para instâncias com até 15 pontos.

3.2.2 Algoritmo Genético

Cada indivíduo representa uma permutação direta dos POIs. A população inicial é gerada aleatoriamente. A função de aptidão é o inverso do *makespan*. A seleção de pais usa torneio de tamanho 2. O cruzamento ordenado (OX) preserva a ordem relativa dos pontos e é aplicado com probabilidade 1. A mutação por troca (*swap*) é aplicada com probabilidade 0,05, trocando dois genes aleatórios. A estratégia de elitismo preserva o melhor indivíduo entre gerações.

3.2.3 Otimização por Enxame de Partículas

Cada partícula mantém um vetor real de dimensão n . A rota é obtida ordenando os índices do vetor em ordem crescente (*random keys*). A velocidade é atualizada por:

$$v_{i,d}^{t+1} = w \cdot v_{i,d}^t + c_1 r_1 (p_{i,d} - x_{i,d}^t) + c_2 r_2 (g_d - x_{i,d}^t), \quad (5)$$

em que $w = 0,7$, $c_1 = c_2 = 2$, $r_1, r_2 \sim U(0, 1)$, p_i é a melhor posição da partícula e g a melhor posição global. A posição é atualizada por $x_{i,d}^{t+1} = x_{i,d}^t + v_{i,d}^{t+1}$.

3.2.4 Otimização por Colônia de Formigas

Cada formiga constrói uma rota incrementalmente. No nó j , a probabilidade de escolher o nó k como próximo é:

$$p_{ijk} = \frac{[\tau_{ijk}]^\alpha \cdot [\eta_{ijk}]^\beta}{\sum_{l \in \mathcal{N}} [\tau_{ijl}]^\alpha \cdot [\eta_{ijl}]^\beta}, \quad (6)$$

em que τ_{ijk} é o feromônio associado à transição $i \rightarrow j \rightarrow k$, $\eta_{ijk} = 1/c_{i,j,k}$ é a informação heurística, $\alpha = 1$ e $\beta = 2$ controlam a importância relativa, e $\mathcal{N}$ é o conjunto de nós ainda não visitados. Após todas as formigas construírem suas rotas, o feromônio é atualizado por:

$$\tau_{ijk} \leftarrow (1 - \rho) \cdot \tau_{ijk} + \sum_{k=1}^m \Delta \tau_{ijk}^k, \quad \Delta \tau_{ijk}^k = \begin{cases} Q/L_k, & \text{se a formiga } k \text{ usou } (i, j, k), \\ 0, & \text{caso contrário,} \end{cases} \quad (7)$$

com $\rho = 0,5$ e $Q = 100$.

3.3 Desenho Experimental

Todas as meta-heurísticas usam população 100 e 100 iterações. A busca exaustiva não tem hiperparâmetros. Instâncias são geradas com coordenadas aleatórias em $[-1, 1]$, com tamanhos 10, 11, 12, 13, 14, 15, 20, 30, 50 e 100 pontos, três instâncias por tamanho. Para cada combinação instância-método, executam-se 55 sementes (GA, PSO, ACO) ou uma execução determinística (busca exaustiva). O total de execuções é de 5029 arquivos de resumo registrados.

Experimentos e Análise dos Resultados

Este capítulo descreve o desenho experimental usado para comparar a busca exaustiva, GA, PSO e ACO, e apresenta a análise quantitativa dos resultados consolidados a partir dos 5029 arquivos de resumo. A formulação do problema e as adaptações específicas de cada método foram detalhadas no Capítulo 3.

4.1 Método para a Avaliação

Todas as execuções avaliam a mesma função objetivo, denominada *makespan*. Uma solução é uma sequência com todos os pontos de interesse, excluindo a origem. O drone parte do nó 0, visita os pontos na ordem indicada e retorna ao nó 0. O custo acumulado usa uma estrutura tridimensional $G[i][j][k]$, que representa o tempo para seguir ao próximo nó considerando o nó anterior, o nó atual e o nó seguinte.

As instâncias são geradas a partir de pontos no plano bidimensional, com coordenadas aleatórias no intervalo $[-1, 1]$. O tempo de deslocamento considera a distância euclidiana e uma penalização angular normalizada pelo ângulo entre o vetor de chegada e o vetor de saída. Essa modelagem aproxima o problema de uma rota de patrulhamento em que manobras mais bruscas aumentam o tempo total.

A avaliação usa três grupos de arquivos produzidos automaticamente pela ferramenta: arquivos de resumo, arquivos de evolução e arquivos de tempo. O resumo contém método, instância, semente, parâmetros, melhor *makespan*, melhor sequência, número de iterações e quantidade de avaliações. A evolução registra cada melhoria observada durante a execução. O arquivo de tempo separa carregamento da instância, otimização, serialização e duração total.

O inventário atual contém 5029 execuções registradas: 1650 para GA, 1650 para PSO, 1650 para ACO e 79 para busca exaustiva. As meta-heurísticas foram executadas em 30 instâncias, com tamanhos de 10, 11, 12, 13, 14, 15, 20, 30, 50 e 100 pontos, três instâncias por tamanho. A busca exaustiva foi aplicada às instâncias menores, nas quais a enumeração completa ainda é viável.

Tabela 1 – Métodos implementados e parâmetros usados nos experimentos.

Método	Representação	Parâmetros principais
Busca exaustiva	Permutação direta	Sem hiperparâmetros
GA	Permutação direta	população=100, iterações=100, elitismo=1, mutação=0,05
PSO	Chaves aleatórias	população=100, iterações=100, $w = 0,7$, $c_1 = 2$, $c_2 = 2$
ACO	Construção probabilística	população=100, iterações=100, $\alpha = 1$, $\beta = 2$, $\rho = 0,5$, $Q =$

4.2 Experimentos

Uma interface de linha de comando implementada em Go executou os experimentos. Cada execução recebe uma instância em formato de grafo, uma semente e um diretório de resultados. O identificador da execução combina instância, método, semente e um hash dos parâmetros, o que reduz o risco de sobrescrever resultados de configurações diferentes.

Para as meta-heurísticas, cada execução usa população 100 e 100 iterações. Como GA, PSO e ACO são métodos estocásticos, a repetição por diferentes sementes é necessária para avaliar estabilidade. Para a busca exaustiva, a semente é registrada apenas por padronização do formato de saída, pois o método não usa aleatoriedade na geração das soluções.

4.3 Avaliação dos Resultados

Esta seção consolida os 5029 arquivos de resumo em estatísticas descritivas por instância e método. A análise cobre três aspectos: qualidade da solução em relação ao ótimo conhecido (nas instâncias pequenas), desempenho relativo entre meta-heurísticas (em todas as instâncias), e comportamento de convergência ao longo das iterações.

4.3.1 Instâncias Pequenas: Comparação com o Ótimo

Nas instâncias de tamanho 10 a 15, a busca exaustiva fornece o valor ótimo global. A Tabela 2 apresenta o melhor makespan encontrado por cada método e o gap percentual em relação ao ótimo.

Entre as meta-heurísticas, o GA destaca-se por encontrar o ótimo global em todas as 18 instâncias no seu melhor caso. O PSO também encontra o ótimo na maioria das instâncias, mas apresenta gap médio de 0,5% a 2% nas instâncias maiores (13–15 nós). O ACO apresenta os maiores gaps, chegando a 14,72% na instância 15c, e seu melhor caso frequentemente fica 4% a 8% acima do ótimo mesmo em instâncias de 11–12 nós.

A variabilidade também difere entre os métodos. O GA apresenta desvio padrão tipicamente entre 0,00 e 0,44, indicando que todas as 55 sementes convergem para soluções próximas ao ótimo. O PSO apresenta desvios de 0,31 a 1,00, refletindo a dependência da qualidade da representação por chaves aleatórias. O ACO apresenta desvios inter-

mediários (0,23 a 0,67), mas com média mais distante do ótimo, indicando convergência prematura para soluções subótimas consistentes. Esse padrão está alinhado com os estudos comparativos da literatura: Haroun et al. (HAROUN; JAMAL; HICHAM, 2015) e Wu (WU, 2020) também relataram sensibilidade do ACO à parametrização e maior estabilidade do GA em instâncias de médio porte.

Tabela 2 – Melhor makespan e gap percentual em relação ao ótimo (busca exaustiva).

Instância	BF (ótimo)	GA	PSO	ACO
10a	10,94	10,94 (0,00%)	10,94 (0,00%)	10,95 (0,07%)
10b	8,96	8,96 (0,00%)	8,96 (0,00%)	9,12 (1,74%)
10c	11,07	11,07 (0,00%)	11,07 (0,00%)	11,07 (0,00%)
11a	11,62	11,62 (0,00%)	11,62 (0,00%)	12,08 (4,04%)
11b	10,74	10,74 (0,00%)	10,74 (0,00%)	11,26 (4,86%)
11c	12,51	12,51 (0,00%)	12,51 (0,00%)	13,30 (6,30%)
12a	13,11	13,11 (0,00%)	13,11 (0,00%)	13,90 (6,02%)
12b	12,68	12,68 (0,00%)	12,86 (1,48%)	12,72 (0,32%)
12c	12,67	12,67 (0,00%)	12,67 (0,00%)	13,43 (5,96%)
13a	13,68	13,68 (0,00%)	13,68 (0,00%)	14,79 (8,16%)
13b	13,96	13,96 (0,00%)	14,03 (0,44%)	14,15 (1,32%)
13c	13,89	13,89 (0,00%)	14,02 (0,98%)	14,48 (4,22%)
14a	13,54	13,54 (0,00%)	13,74 (1,50%)	13,94 (2,97%)
14b	14,20	14,20 (0,00%)	14,29 (0,69%)	14,80 (4,27%)
14c	14,91	14,91 (0,00%)	15,13 (1,48%)	15,61 (4,70%)
15a	13,72	13,72 (0,00%)	14,09 (2,67%)	14,07 (2,53%)
15b	16,01	16,01 (0,00%)	16,58 (3,56%)	18,12 (13,19%)
15c	15,20	15,20 (0,00%)	15,99 (5,19%)	17,43 (14,72%)

4.3.2 Instâncias Maiores: Desempenho Relativo

Para as instâncias de 20 a 100 nós, não há ótimo conhecido por busca exaustiva. A comparação é relativa entre os três métodos. A Tabela 3 apresenta os melhores valores encontrados.

O GA mantém a superioridade observada nas instâncias pequenas. Na instância 100a, por exemplo, o GA encontra makespan 114,65, contra 134,81 do PSO e 146,95 do ACO – uma diferença de aproximadamente 15% e 22%, respectivamente. Essa vantagem se mantém consistentemente em todas as 12 instâncias maiores.

O PSO ocupa a segunda posição na maioria das instâncias. O ACO apresenta os maiores makespans, ficando tipicamente 10% a 20% acima do GA. Esse resultado sugere que o modelo de feromônio tridimensional do ACO, embora conceitualmente adequado ao problema, sofre com a diluição da informação de feromônio no espaço de busca tridimensional, reduzindo a eficácia da construção probabilística de rotas.

Tabela 3 – Melhor makespan por instância nas execuções sem busca exaustiva.

Instância	GA	PSO	ACO
20a	19,34	21,31	24,83
20b	19,58	22,75	24,41
20c	18,87	20,90	21,14
30a	28,83	34,83	39,14
30b	28,79	34,67	36,46
30c	29,06	34,56	38,21
50a	46,37	59,83	65,80
50b	49,18	66,30	72,41
50c	49,36	63,77	70,32
100a	114,65	134,81	146,95
100b	111,76	136,52	145,51
100c	116,66	136,92	150,39

4.3.3 Convergência e Estabilidade

Os registros de evolução permitem analisar a velocidade de convergência. O GA tende a encontrar melhorias nas primeiras iterações e converge gradualmente, com melhorias esporádicas ao longo das 100 iterações. O PSO apresenta atualizações mais frequentes no início, mas a qualidade da solução estagna mais cedo, possivelmente devido à limitação da representação por chaves aleatórias em capturar a estrutura combinatória do problema. O ACO converge rapidamente para uma solução nas primeiras iterações e raramente apresenta melhorias após as primeiras 20–30 iterações, consistente com a hipótese de convergência prematura.

A dispersão entre sementes (medida pelo desvio padrão do melhor makespan) confirma esses padrões. O GA é o método mais estável, com desvio tipicamente entre 0,1 e 4,3, crescendo com o tamanho da instância. O PSO apresenta a maior dispersão (desvios de 0,3 a 3,4), refletindo a variabilidade introduzida pela representação contínua. O ACO apresenta dispersão moderada (0,2 a 1,7), coerente com uma convergência consistente para soluções de qualidade similar, ainda que subótimas.

4.3.4 Limitações Metodológicas

Uma limitação metodológica já identificada é a necessidade de revisar a consistência entre a documentação da estrutura do grafo e a ordem dos índices usada na geração dos custos. A função objetivo interpreta o grafo como $G[\text{anterior}][\text{atual}][\text{próximo}]$; a etapa de geração da instância deve ser validada antes de se atribuir significado físico definitivo à penalização de curva. Essa verificação não impede a comparação interna entre métodos, pois todos otimizam a mesma função, mas afeta a interpretação do modelo como aproximação de voo real.

Além disso, os hiperparâmetros das meta-heurísticas foram fixados sem ajuste fino: população 100 e 100 iterações para todos os métodos. Valores diferentes poderiam alterar o ordenamento relativo, especialmente para o ACO, cujos parâmetros α , β , ρ e Q não foram otimizados por instância. Essa limitação é comum em estudos comparativos e, como observam Chandra e Naro (CHANDRA; NARO, 2022) e Almufti e Shaban (ALMUFTI; SHABAN, 2025), o ajuste de parâmetros por instância pode produzir ordenações diferentes das obtidas com valores fixos.

Conclusão

Este trabalho investigou o problema de otimização de rotas para patrulhamento com drones, modelado como uma variante do Problema do Caixeiro Viajante com custo dependente da sequência de visita. Quatro métodos foram implementados e comparados: busca exaustiva (referência exata), Algoritmo Genético, Otimização por Enxame de Partículas e Otimização por Colônia de Formigas. Os experimentos totalizaram 5029 execuções em 30 instâncias entre 10 e 100 pontos.

5.1 Principais Contribuições

Os resultados confirmam a hipótese principal: as meta-heurísticas produzem rotas de baixo *makespan* com custo computacional muito inferior ao da enumeração completa. O GA destaca-se como o método mais eficaz: encontrou o ótimo global em todas as 18 instâncias pequenas (10–15 pontos) no seu melhor caso, com desvio padrão máximo de 0,44, e manteve vantagem consistente nas instâncias maiores, com *makespan* até 22% inferior ao do ACO na instância 100a.

O PSO ocupou a segunda posição na maioria das instâncias, mas apresentou maior variabilidade entre sementes, refletindo a dependência da representação por chaves aleatórias. O ACO apresentou os maiores *gaps* em relação ao ótimo, com convergência prematura nas primeiras 20–30 iterações. Esse resultado sugere que o feromônio tridimensional sofre diluição no espaço de busca, reduzindo a eficácia da construção probabilística.

A hipótese secundária também se confirmou: GA e ACO, por operarem diretamente sobre permutações, exploram a estrutura combinatória do problema de forma mais natural que o PSO. No entanto, o ACO não se beneficiou dessa vantagem devido à dispersão do feromônio no tensor tridimensional.

As contribuições específicas incluem: (i) a formalização do modelo com custo dependente da sequência e penalização angular; (ii) a implementação comparativa de quatro métodos sobre a mesma função objetivo; (iii) a base experimental com 5029 execuções

e saídas estruturadas; (iv) a análise empírica detalhada de convergência, estabilidade e escalabilidade.

5.2 Limitações

Três limitações metodológicas devem ser consideradas. Primeiro, a consistência entre a documentação do tensor $G[i][j][k]$ e a ordem dos índices na geração dos custos precisa ser validada antes de atribuir significado físico definitivo à penalização de curva. Embora isso não afete a comparação entre métodos, limita a interpretação do modelo como simulação de voo real. Segundo, os hiperparâmetros foram fixados sem ajuste fino (população 100, 100 iterações para todos). O ACO, em particular, poderia se beneficiar de variações em α , β , ρ e Q . Terceiro, as instâncias são sintéticas com coordenadas aleatórias, o que não reflete necessariamente a geometria de cenários reais de patrulhamento.

5.3 Trabalhos Futuros

Diversas direções de continuidade são possíveis. O ajuste fino de hiperparâmetros por instância poderia alterar o ordenamento relativo entre os métodos. A implementação de variantes mais avançadas de cada meta-heurística, como o MMAS para ACO (com limites $\tau_{\min}$ e $\tau_{\max}$) ou o operador EAX para GA, pode melhorar a qualidade das soluções. A hibridização entre métodos, como no esquema GA-ACO em dois estágios (SINGH; SINGH; CHAURASIA, 2024), representa uma direção promissora.

No horizonte mais amplo, a integração de aprendizado profundo com meta-heurísticas swarm – como DeepACO (YE et al., 2023), NeuFACO (YE et al., 2025) e PGACO/PPOACO (SHEPPARD et al., 2024) – constitui o estado-da-arte em otimização combinatória neural. A aplicação dessas abordagens ao modelo com custo dependente da sequência é uma extensão natural. Por fim, a validação do modelo em cenários reais de patrulha, com dados geográficos e restrições operacionais de VANTs, permitiria avaliar o impacto prático das soluções encontradas.

5.4 Contribuições em Produção Bibliográfica

Este trabalho gerou uma base de código aberta para experimentos com meta-heurísticas em problemas de roteamento, documentada no repositório associado a esta monografia.

Referências

- AGATZ, N.; BOUMAN, P.; SCHMIDT, M. Optimization approaches for the traveling salesman problem with drone. **Transportation Science**, v. 52, n. 4, p. 965–981, 2018.
- AHMED, G.; SHELAMI, T. Receding horizon and optimization-based control for UAV path planning with collision avoidance. **Procedia Computer Science**, v. 251, p. 15–22, 2024.
- ALEXANDER, A.; SRIWINDONO, H. The comparison of genetic algorithm and ant colony optimization in completing travelling salesman problem. In: **Proceedings of the 2nd International Conference of Science and Technology for the Internet of Things (ICSTI 2019)**. [S.l.]: EAI, 2020.
- ALMUFTI, S. M.; SHABAN, A. A. Comparative analysis of metaheuristic algorithms for solving the travelling salesman problems. **International Journal of Scientific World**, v. 11, n. 2, p. 26–30, 2025.
- APPLEGATE, D. L. et al. **The Traveling Salesman Problem: A Computational Study**. Princeton: Princeton University Press, 2006.
- BEAN, J. C. Genetic algorithms and random keys for sequencing and optimization. **ORSA Journal on Computing**, v. 6, n. 2, p. 154–160, 1994.
- BLUM, C. Ant colony optimization: Introduction and recent trends. **Physics of Life Reviews**, v. 2, n. 4, p. 353–373, 2005.
- BOCK, S. et al. A survey on the traveling salesman problem and its variants in a warehousing context. **European Journal of Operational Research**, v. 322, n. 1, p. 1–14, 2025.
- CHANDRA, A.; NARO, A. A comparative study of metaheuristics methods for solving traveling salesman problem. **International Journal of Information Science and Technology**, v. 6, n. 2, p. 1–7, 2022.
- CLERC, M. Discrete particle swarm optimization, illustrated by the traveling salesman problem. In: ONWUBOLU, G. C.; BABU, B. V. (Ed.). **New Optimization Techniques in Engineering**. [S.l.]: Springer, 2004. (Studies in Fuzziness and Soft Computing, v. 141), p. 219–239.

DELL'AMICO, M.; MONTEMANNI, R.; NOVELLANI, S. Modeling the flying sidekick traveling salesman problem with multiple drones. **Networks**, v. 78, n. 3, p. 303–327, 2021.

_____. Exact models for the flying sidekick traveling salesman problem. **International Transactions in Operational Research**, v. 29, n. 3, p. 1360–1393, 2022.

DORIGO, M.; BLUM, C. Ant colony optimization theory: A survey. **Theoretical Computer Science**, v. 344, n. 2–3, p. 243–278, 2005.

DORIGO, M.; GAMBARDELLA, L. M. Ant colonies for the travelling salesman problem. **BioSystems**, v. 43, n. 2, p. 73–81, 1997.

DORIGO, M.; MANIEZZO, V.; COLORNI, A. Ant system: Optimization by a colony of cooperating agents. **IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics**, v. 26, n. 1, p. 29–41, 1996.

DORIGO, M.; STÜTZLE, T. **Ant Colony Optimization**. Cambridge: MIT Press, 2004.

FREITAS, J. C. de; PENNA, P. H. V. A variable neighborhood search for flying sidekick traveling salesman problem. **International Transactions in Operational Research**, v. 27, n. 1, p. 267–290, 2020.

GAREY, M. R.; JOHNSON, D. S. **Computers and Intractability: A Guide to the Theory of NP-Completeness**. San Francisco: W. H. Freeman, 1979.

GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. Reading: Addison-Wesley, 1989.

HALIM, A. H.; ISMAIL, I. Combinatorial optimization: Comparison of heuristic algorithms in travelling salesman problem. **Archives of Computational Methods in Engineering**, v. 26, n. 2, p. 367–380, 2019.

HAROUN, S. A.; JAMAL, B.; HICHAM, E. H. A performance comparison of GA and ACO applied to TSP. **International Journal of Computer Applications**, v. 117, n. 20, p. 28–35, 2015.

HOLLAND, J. H. **Adaptation in Natural and Artificial Systems**. Ann Arbor: University of Michigan Press, 1975.

HOSSAIN, M. A. A.; ACAR, Z. Y. Comparison of new and old optimization algorithms for traveling salesman problem on small, medium, and large-scale benchmark instances. **Bitlis Eren Üniversitesi Fen Bilimleri Dergisi**, v. 13, n. 1, p. 216–231, 2024.

KAPPAGANTULA, S. et al. DPSO-Q: A reinforcement learning-enhanced swarm algorithm for solving the traveling salesman problem. **International Journal of Intelligent Systems**, v. 2025, p. 8918171, 2025.

KENNEDY, J.; EBERHART, R. C. Particle swarm optimization. In: **Proceedings of ICNN'95 – International Conference on Neural Networks**. [S.l.]: IEEE, 1995. p. 1942–1948.

- LAWLER, E. L. et al. (Ed.). **The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization**. New York: Wiley, 1985.
- LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, v. 21, n. 2, p. 498–516, 1973.
- LIU, X.; CHEN, Y.; ZHANG, M. Automated design of state transition rules in ant colony optimization by genetic programming. **Memetic Computing**, 2025.
- MURRAY, C. C.; CHU, A. G. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. **Transportation Research Part C: Emerging Technologies**, v. 54, p. 86–109, 2015.
- nAGA, P. L. et al. Genetic algorithms for the travelling salesman problem: A review of representations and operators. **Artificial Intelligence Review**, v. 13, n. 2, p. 129–170, 1999.
- NAGATA, Y. Edge assembly crossover for the traveling salesman problem. In: **Proceedings of the 6th European Conference on Evolutionary Computation in Combinatorial Optimization (EvoCOP 2006)**. [S.l.]: Springer, 2006. (LNCS, v. 3906), p. 146–158.
- POP, P. C. et al. A comprehensive survey on the generalized traveling salesman problem. **European Journal of Operational Research**, v. 314, n. 3, p. 819–835, 2024.
- POTVIN, J.-Y. Genetic algorithms for the traveling salesman problem. **Annals of Operations Research**, v. 63, n. 3, p. 337–370, 1996.
- RAJAN, S.; SUNDAR, K.; GAUTAM, N. Routing problem for unmanned aerial vehicle patrolling missions – a progressive hedging algorithm. **Computers & Operations Research**, v. 142, p. 105702, 2022.
- RAJWAR, K.; DEEP, K.; DAS, S. An exhaustive review of the metaheuristic algorithms for search and optimization: Taxonomy, applications, and open challenges. **Artificial Intelligence Review**, v. 56, n. 11, p. 13187–13257, 2023.
- SHEPPARD, J. et al. Policy gradient and experience replay in ant colony optimization. **arXiv preprint arXiv:2410.07388**, 2024.
- SINGH, D. R.; SINGH, M. K.; CHAURASIA, S. N. An efficient hybrid algorithm with novel inver-over operator and ant colony optimization for traveling salesman problem. **arXiv preprint arXiv:2412.00340**, 2024.
- STÜTZLE, T.; HOOS, H. H. MAX-MIN ant system. **Future Generation Computer Systems**, v. 16, n. 8, p. 889–914, 2000.
- TOAZA, B.; ESZTERGÁR-KISS, D. A review of metaheuristic algorithms for solving TSP-based scheduling optimization problems. **Applied Soft Computing**, v. 148, p. 110908, 2023.
- VANHOVE, S.; FACK, V. Route planning with turn restrictions: A computational experiment. **Operations Research Letters**, v. 40, n. 5, p. 342–348, 2012.

WADI, A. H. A.; UMAR, S. U. Charting new routes: Comparing swarm-based approaches to the traveling salesman problem. **International Journal of Computational Methods and Experimental Measurements**, v. 13, n. 2, p. 371–379, 2025.

WINTER, S. Modeling costs of turns in route planning. **GeoInformatica**, v. 6, n. 4, p. 345–360, 2002.

WU, Z. A comparative study of solving traveling salesman problem with genetic algorithm, ant colony algorithm, and particle swarm optimization. In: **Proceedings of the 2020 2nd International Conference on Robotics Systems and Vehicle Technology (RSVT '20)**. [S.l.]: ACM, 2020. p. 95–99.

YE, H. et al. DeepACO: Neural-enhanced ant colony optimization for combinatorial optimization. **arXiv preprint arXiv:2309.14032**, 2023.

_____. NeuFACO: Neural focused ant colony optimization for traveling salesman problem. **arXiv preprint arXiv:2503.08812**, 2025.

Apêndices

Anexos

